

Engenharia de Prompts · Entendendo os Parâmetros

```
from dotenv import load_dotenv
from groq import Groq
import os
load_dotenv()

client = Groq(
    api_key=os.environ.get("GROQ_API"), # This is the default and can be omitted
)
```

Parâmetros Importantes

- **temperature:** controla a aleatoriedade das respostas. Valores entre 0.0 e 1.0 são aceitos.
- **max_completion_tokens:** define o número máximo de tokens que podem ser gerados pelo modelo. Esse parâmetro substitui o max_tokens em desuso.
- **top_p:** controla a diversidade via "nucleus sampling" — em torno de 0.5 significa considerar apenas os tokens que somam 50% da probabilidade.

Usando Temperature

```
prompt = "Escreva uma frase poética sobre o anoitecer"

# Temperature baixa (0.0) - Mais determinística
print("Temperature 0.0 (Determinística):")
response_low = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": "Você é um poeta minimalista que descreve a natureza de forma simples e direta."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.0
)
print(f"Resposta: {response_low.choices[0].message.content}\n")
```

```
# Temperature média (0.7) - Equilibrada
print("Temperature 0.7 (Equilibrada):")
response_medium = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": "Você é um poeta clássico que mistura metáforas e imagens da natureza."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.7
)
print(f"Resposta: {response_medium.choices[0].message.content}\n")
```

```
# Temperature alta (1.5) - Mais criativa
print("Temperature 1.5 (Criativa):")
response_high = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": "Você é um poeta surrealista que usa imagens improváveis e
```

```
linguagem ousada."},
    {"role": "user", "content": prompt}
],
temperature=1.5
)
print(f"Resposta: {response_high.choices[0].message.content}")
```

Usando top_p

```
prompt = "Complete a frase: 'A inteligência artificial no futuro será...'"
print("=== Comparação Top_p ===\n")

# Top_p baixo (0.1) - Mais focado
print("Top_p 0.1 (Focado):")
response_focused = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": "Você é um futurista que faz previsões sérias e objetivas sobre tecnologia."},
        {"role": "user", "content": prompt}
    ],
    temperature=1.0, # fixo para comparar só o efeito do top_p
    top_p=0.1
)
print(f"Resposta: {response_focused.choices[0].message.content}\n")
```

```
# Top_p médio (0.7) - Equilibrado
print("Top_p 0.7 (Equilibrado):")
response_balanced = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": "Você é um futurista que mistura objetividade com toques criativos em previsões."},
        {"role": "user", "content": prompt}
    ],
    temperature=1.0,
    top_p=0.7
)
print(f"Resposta: {response_balanced.choices[0].message.content}\n")
```

```
# Top_p alto (0.9) - Mais diverso
print("Top_p 0.9 (Diverso):")
response_diverse = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": "Você é um futurista visionário que cria previsões ousadas e imaginativas."},
        {"role": "user", "content": prompt}
    ],
    temperature=1.0,
    top_p=0.9
)
print(f"Resposta: {response_diverse.choices[0].message.content}")
```